README DEL PROYECTO

Sistema de Gestión y Venta de Gas

"Distribuidora de Gas El Volcán"

Asignatura	DSY1104 — Desarrollo Fullstack II
Integrantes	Nicolás Arancibia Pablo Gómez Benjamín González
Profesor(a)	Hernán Saavedra
Versión	2.1

CHILLÁN · REGIÓN DE ÑUBLE · CHILE

Distribuidora de Gas El Volcán — Versión React

Aplicación web **SPA (Single Page Application)** de venta y distribución de gas GLP y accesorios, construida con **React + Vite + React Bootstrap + React Router**. Es la **Versión 2** del proyecto semestral de Desarrollo Full Stack II (DSY1104): reemplaza el sitio estático (HTML/CSS/JS) de la Versión 1 por una aplicación React moderna con componentes, `props`, `state` y pruebas unitarias.

Los datos se guardan en `localStorage` (navegador) hasta que existan los microservicios en Spring Boot.

Stack

Tecnología	Versión	Para qué
Vite	8.3	Creación, servidor de desarrollo y build
React / React DOM	19.2	Componentes y SPA
react-router-dom	7.18	Rutas de la aplicación
react-bootstrap	2.10	Componentes de interfaz
Bootstrap	5.3.8	Estilos CSS
bootstrap-icons	1.13	Íconos
oxlint	1.81	Linter (`npm run lint`)
Vitest	3.2	Pruebas unitarias (`npm test`)
@testing-library/react + jest-dom + user-event	16.3 / 6.6 / 14.6	Renderizado, matchers DOM e interacción con el usuario
jsdom	26.1	Entorno navegador simulado para las pruebas
@vitest/coverage-v8	3.2	Reporte de cobertura (`npm run test:coverage`)

Cómo correrlo

```
npm install           # instala dependencias (solo la primera vez)
npm run dev           # servidor de desarrollo → http://localhost:5173
npm run lint          # oxlint: 0 errores
npm run build         # build de producción → carpeta dist/
npm test              # ejecuta las 10 pruebas unitarias (una sola vez)
npm run test:watch    # pruebas en modo vigilancia (se re-ejecutan al guardar)
npm run test:coverage # pruebas + reporte de cobertura en coverage/
```

> En PowerShell usar `npm.cmd` (la política de ejecución puede bloquear `npm.ps1`). Las pruebas corren en

> **jsdom** (DOM simulado de Node), sin necesidad de abrir un navegador.

Credenciales de prueba (administrador): `admin@duoc.cl` / `Admin123`

Estructura del proyecto

```
avance entrega/
│
├── README.md                        → este archivo
├── DOCUMENTACION.md                 → explicación detallada de cada parte y su lógica
├── ERS_ACTUALIZADO.md               → Especificación de Requerimientos de Software
├── DOCUMENTO_COBERTURA_TESTING.md   → informe de las pruebas y la cobertura
├── DEPLOY.md                        → guía de despliegue en AWS EC2 + Nginx
├── BITACORA_DEPLOY_AWS.md           → bitácora real del despliegue (evidencia, 10-10-2026)
├── nginx.conf                       → configuración de Nginx usada en el deploy
├── index.html                       → punto de montaje de la SPA (lang="es")
├── package.json                     → dependencias y comandos
├── vite.config.js                   → configuración de Vite (incluye el bloque test de Vitest)
├── .oxlintrc.json                   → reglas del linter
├── .gitignore                       → archivos que NO se suben (node_modules, dist, coverage)
│
├── public/                          → archivos servidos tal cual
│   ├── IMAGENES/                    → imágenes de productos y del sitio
│   ├── favicon.svg
│   └── icons.svg
│
└── src/
    ├── main.jsx                     → punto de entrada: monta React y carga los CSS
    ├── index.css                    → estilos globales mínimos
    ├── estilos.css                  → hoja de estilos heredada de la Versión 1
    ├── setupTests.js                → configuración global de Vitest (jest-dom)
    ├── App.jsx                      → rutas de la SPA + estado global del carrito
    │
    ├── datos/
    │   └── productos.js             → "base de datos" local: los 14 productos de gas
    │
    ├── utils/                       → lógica de negocio pura y reutilizable (testeable)
    │   ├── validaciones.js          → validaciones, regiones/comunas y helpers de localStorage
    │   ├── carrito.js               → operaciones del carrito (agregar, quitar, total)
    │   ├── productos.js             → catálogo real (base + ediciones + productos creados)
    │   ├── validaciones.test.js     → 2 pruebas
    │   ├── carrito.test.js          → 3 pruebas
    │   └── productos.test.js        → 1 prueba
    │
    ├── components/                  → piezas reutilizables
    │   ├── Navbar.jsx               → barra de navegación (todas las páginas públicas)
    │   ├── Footer.jsx               → pie de página
    │   ├── BotonTienda.jsx          → botón flotante "encuentra tu tienda"
    │   ├── LayoutPublico.jsx        → layout público: BotonTienda + Navbar + contenido + Footer
    │   ├── Producto.jsx             → tarjeta de producto (recibe props)
```

```
│   ├── Producto.test.jsx       → 1 prueba (componente)
│   └── Navbar.test.jsx         → 1 prueba (componente)
│
├── pages/                      → una página por cada ruta
│   ├── Inicio.jsx              → portada
│   ├── Productos.jsx           → catálogo con filtro por categoría (?categoria=)
│   ├── DetalleProducto.jsx     → ficha de un producto (/producto/:id)
│   ├── Carrito.jsx             → carrito con cantidades, subtotales y total
│   ├── Ventas.jsx              → checkout: dirección, comuna y método de pago
│   ├── Login.jsx               → acceso (formulario controlado + validación)
│   ├── Registro.jsx            → creación de cuenta de cliente
│   ├── Nosotros.jsx            → historia y compromiso (blogs expandibles)
│   ├── Consejos.jsx            → consejos de seguridad y uso del gas
│   ├── Tiendas.jsx             → sucursales y mapa
│   ├── Contacto.jsx            → formulario de contacto
│   ├── Nosotros.test.jsx       → 1 prueba (estado)
│   ├── Login.test.jsx          → 1 prueba (formulario interactivo)
│   └── admin/                  → panel de administración (layout propio)
│       ├── AdminLayout.jsx     → menú lateral, sesión y cierre de sesión
│       ├── Dashboard.jsx       → indicadores, órdenes y stock crítico
│       ├── Ordenes.jsx         → panel de órdenes (datos de ejemplo)
│       ├── Clientes.jsx        → panel de clientes (datos de ejemplo)
│       ├── Reportes.jsx        → panel de reportes (datos de ejemplo)
│       ├── Inventario.jsx      → CRUD de productos
│       ├── ListaUsuarios.jsx   → listado y filtro de usuarios
│       └── NuevoUsuario.jsx    → creación de usuarios
```

Funcionalidades

Parte pública

- **Inicio** con mensaje de bienvenida y acceso al catálogo.
- **Catálogo** con los 14 productos reales (cilindros 5/11/15/45 kg, reguladores, mangueras, accesorios) y precios residencial/comercial.
- **Filtro por categoría** con query string (`/productos?categoria=Reguladores`).
- **Detalle de producto** por ruta dinámica (`/producto/:id`).
- **Carrito** con `+` / `-`, quitar ítems, subtotales y total; persistente en `localStorage`.
- **Checkout (`/ventas`)**: dirección, comuna y método de pago; genera una **orden** y vacía el carrito.
- **Registro** de clientes con validaciones (RUN, correo institucional, teléfono, región/comuna, contraseña, términos).
- **Login** validando contra `localStorage.usuarios` + el admin precargado.
- Páginas informativas **Nosotros**, **Consejos**, **Tiendas** (mapa) y **Contacto**.

Panel de administración (`/admin`)

- **Dashboard** con indicadores (ventas del día, órdenes en ruta, clientes nuevos) y stock crítico.
- **Inventario**: crear, editar y eliminar productos, con validaciones y alertas de stock.
- **Usuarios**: listar y filtrar por rol, y crear nuevos usuarios.

Pruebas

- **10 pruebas unitarias** con **Vitest** (`npm test` , jsdom + React Testing Library) sobre utilidades y componentes.
- Incluyen **validaciones**, **formulario interactivo** (`Login`), **carrito/compras**, **CRUD del catálogo**, props, estado y mocks.
- Reporte de cobertura en `coverage/index.html` (`npm run test:coverage`).

Pruebas y cobertura

Las 10 pruebas se agrupan por tipo (ver detalle y análisis en `DOCUMENTO_COBERTURA_TESTING.md`):

#	Test	Cubre
1	`validaciones.test.js`	RUN, teléfono, correo, dominio y formato de precio
2	`validaciones.test.js`	`validarFormularioRegistro` + `validarFormularioContacto` (errores) y helpers de `localStorage`
3	`carrito.test.js`	`agregarItem` / `incrementarItem` (sin mutar)
4	`carrito.test.js`	`decrementarItem` / `eliminarItem` / `contarItems`
5	`carrito.test.js`	`obtenerItemsCompletos` / `calcularTotal` (residencial y comercial)
6	`productos.test.js`	`aplicarEdiciones` / `catalogoActual` / `productosConCreados` (CRUD)
7	`Producto.test.jsx`	props en el DOM y callback `onAgregar` (mock con `vi.fn()`)
8	`Navbar.test.jsx`	enlaces y prop `totalItems`
9	`Nosotros.test.jsx`	estado `useState` (expandir/contraer blog)
10	`Login.test.jsx`	formulario interactivo: escribes y valida el correo (user-event)

Cobertura obtenida: 100 % sentencias · 100 % ramas · 100 % funciones · 100 % líneas (sobre los módulos que participan en las pruebas).

Documentación relacionada

- `DOCUMENTACION.md` — explicación técnica de cada componente, página y decisión de diseño.
- `ERS_ACTUALIZADO.md` — Especificación de Requerimientos de Software (requisitos funcionales y no funcionales).
- `DOCUMENTO_COBERTURA_TESTING.md` — inventario de pruebas, resultados y análisis de cobertura.

Estado y próximos pasos

Implementado en esta entrega (frontend React): catálogo, carrito, checkout, cuentas, contacto y panel de administración, con persistencia en `localStorage` y pruebas unitarias.

Planificado (etapas posteriores):

- Backend con **microservicios Spring Boot** (API REST/JSON) y base de datos **MySQL**, reemplazando `localStorage`.
- **Autenticación con tokens** y control de acceso por roles (**RBAC**) verificado en el servidor.
- Seguimiento de pedidos en **tiempo real con mapas** (Leaflet / Google Maps).
- Despliegue en **AWS** con **Docker**.

Duoc UC — Analista Programador | DSY1104 Desarrollo Full Stack II